Course: Individual Software Development Process' 2026

Software Proposal

By UltraSmooth

Chosen Irl Challenge : Project A - Lab Data Management

Jirat Kiattrairong	6810545506
Napakhet Namsrioun	6810545727
Panyasiri Aimngern	6810545743
Panisara Niyathirakul	6810545751

Table of Contents

1. Target Users

- **Ph.D. Students :** Lab members who can view all relevant laboratory information and records, search/filter/sort resources, submit and track bookings or requests for lab resources, and check in/check out resources they have booked.
- **Undergraduate Students :** Lab members with the same permissions as Ph.D. Students - view laboratory information and records, search/filter/ sort resources, submit and track bookings or requests, and check in/check out resources they have booked.
- **Lab Managers / Administrators :** The system's admin role. In addition to all permissions granted to Ph.D./Undergraduate Students, they can create, edit and archive resource records that approve or reject booking/request submissions; report and manage resource issues and maintenance, view and filter the audit log (activity history), view the operations dashboard and manage user accounts and role assignments.
- **Professor :** Can only view all relevant laboratory information and records (resources, bookings/requests, activity history, dashboard), but does not have approval, management, or account-administration privileges.

2. The System Goal

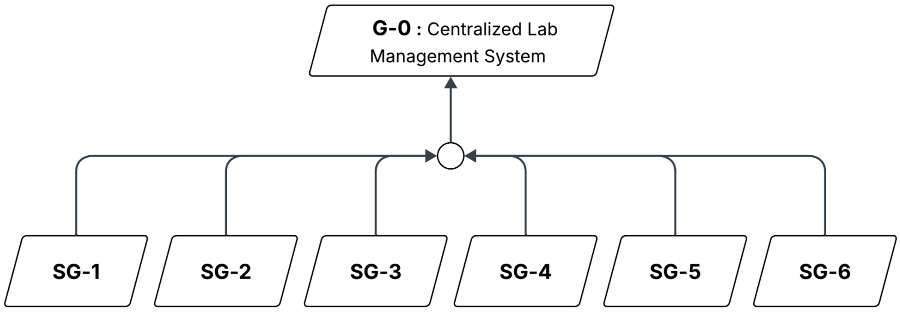
G-0: Provide a centralized and reliable system for managing laboratory resources, activities, and operational information.

As a newly established Software Engineering Laboratory, VASE currently does not have a centralized system for managing its resources and daily operations. The laboratory relies on separate tools and manual processes to manage information such as resources, bookings, requests, maintenance activities, and records. Since these tools are not connected, information can become inconsistent or outdated, making it difficult to track activities, monitor resource usage, and review previous records. Managing these separate tools across different computers can also be inconvenient and time-consuming.

The proposed system aims to bring the laboratory's operational information into one centralized system. It will provide a consistent way to manage and access information related to laboratory resources and activities, while supporting processes such as resource management, booking, requests, maintenance, and activity tracking. The system can also provide an overview of important laboratory information to help staff monitor the current state and usage of resources.

The detailed features and workflows will be refined through requirement gathering with the stakeholder, Aj. Milk. Features that are not necessary for the initial scope, such as advanced financial management or external guest access, can be considered for future versions as the laboratory's requirements develop.

3. Goal Refinement via KAOS:



- **Sub-Goal 1 (SG-1) — Resource Information**

Description: Achieve: Keep laboratory resource information accurate, current, and accessible.

Rationale: Centralizing resource information helps users access accurate and up-to-date resource details.

- **Sub-Goal 2 (SG-2) — Booking & Request Management**

Description: Achieve: Enable laboratory bookings and requests to be submitted, reviewed, approved or rejected, and tracked.

Rationale: A centralized process makes bookings and requests easier to manage, review, and track.

- **Sub-Goal 3 (SG-3) — Issues & Maintenance**

Description: Achieve: Ensure resource issues and maintenance activities are reported, tracked, and reflected in resource status.

Rationale: Recording issues and maintenance activities helps staff monitor resource conditions and keep resource status updated

- **Sub-Goal 4 (SG-4) — History & Dashboard**

Description: Maintain: An accurate and reviewable history of laboratory activities and a clear overview of current operations.

Rationale: Centralized activity records and dashboard information help users review past activities and monitor current operations.

- **Sub-Goal 5 (SG-5) — Resource Availability & Controlled Usage**

Description: Avoid: Conflicting bookings and unauthorized resource use during check-in and check-out.

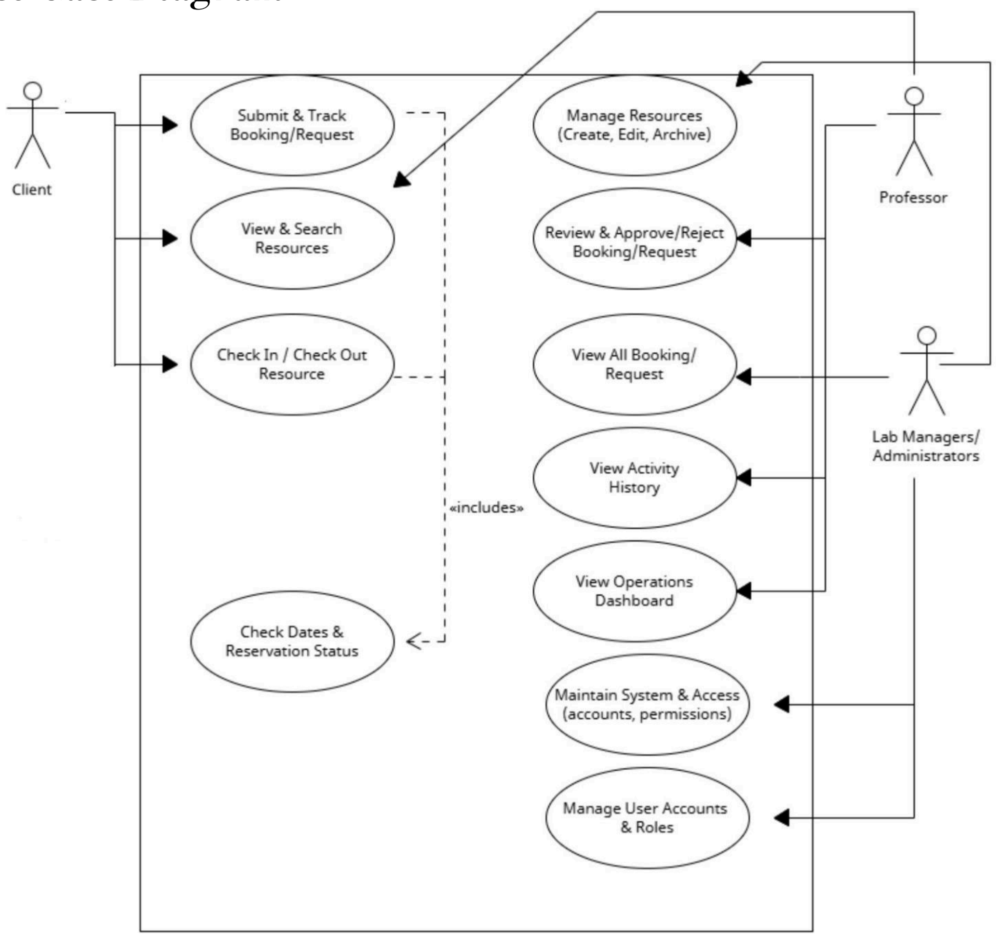
Rationale: Checking resource availability and booking validity helps prevent scheduling conflicts and unauthorized resource use.

- **Sub-Goal 6 (SG-6) — System Access & Role Management**

Description: Achieve: Ensure users access system features and information according to their roles and permissions.

Rationale: Role-based access ensures users can access only the features and information permitted for their responsibilities.

4. Use Case Diagram



5. User Stories

User Story ID	Description	Acceptance Criteria
US-1	As a Ph.D. Student, Undergraduate Student, Lab Manager/Administrator, or Professor, I want to view and search resources so I can find what I need.	Ph.D. Students, Undergraduate Students, Lab Managers/Administrators, and Professors can see resource name, type, category, and status. All four roles can search by keyword and filter/sort by category, location, availability, and status.
US-2	As a Lab Manager/Administrator, I want to manage resource records so information stays accurate.	Lab Managers/Administrators can add, edit, and archive resources, including status, availability, and ownership. Required fields must be filled before saving; missing/invalid data triggers a validation message and blocks saving.
US-3	As a Ph.D. Student or Undergraduate Student, I want to submit a booking/request so I can reserve a resource.	Ph.D. Students and Undergraduate Students can submit a request with resource, date/time, and purpose. The system checks for overlapping bookings and blocks the request if a conflict exists.
US-4	As a Ph.D. Student or Undergraduate Student, I want to track my bookings/requests so I know their status.	Ph.D. Students and Undergraduate Students can see their own requests with current status (Pending/Approved/Rejected). Status updates automatically after a Lab Manager/Administrator's decision.
US-5	As a Lab Manager/Administrator, I want to review and approve/reject requests so I can control resource usage.	Lab Managers/Administrators can view pending requests and approve or reject each one. Status updates immediately and is visible to the requester.
US-6	As a Lab Manager/Administrator, I want to report a resource issue so broken resources are documented and blocked from booking.	Lab Managers/Administrators can log an issue with description, status, and date. Resource status auto-changes to "Maintenance" and is blocked from new bookings.
US-7	As a Lab Manager/Administrator, I want to resolve maintenance records so resource status stays accurate.	Lab Managers/Administrators can update maintenance records and mark issues resolved. Resource status auto-reverts to "Available."

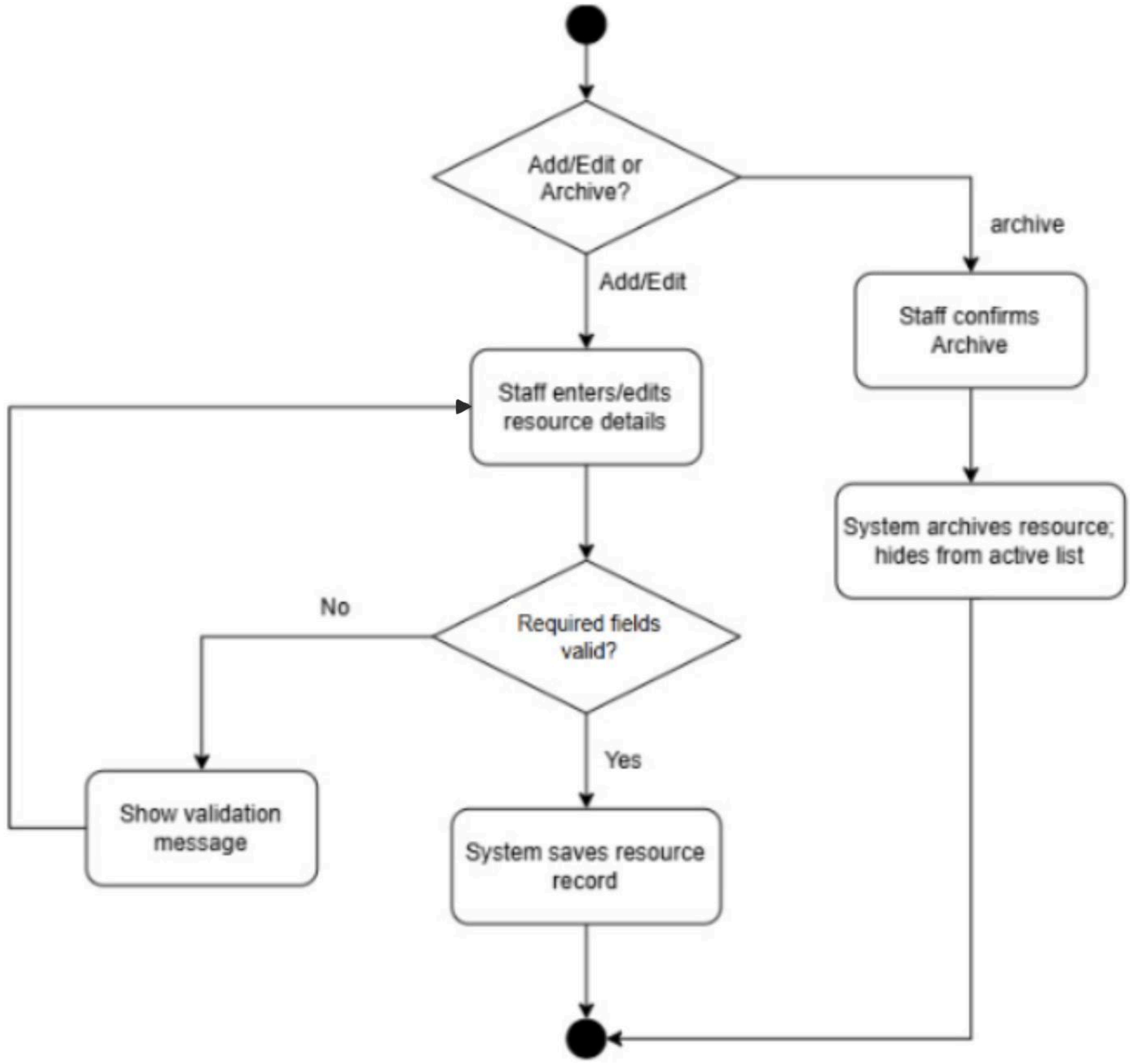
US-8	As a Lab Manager/Administrator or Professor, I want to view and filter the activity log so I can check past actions.	Lab Managers/Administrators and Professors can view the log (action, user, timestamp) and filter by date, user, or action type.
US-9	As a Lab Manager/Administrator or Professor, I want a dashboard summary so I can quickly check lab status.	Lab Managers/Administrators and Professors see total/available resources, active bookings, and open issues on the dashboard. Utilization is shown by category over a selectable time period.
US-10	As a Ph.D. Student or Undergraduate Student, I want to check in/out a resource so its status updates automatically.	Ph.D. Students and Undergraduate Students can scan/enter a resource code. The system verifies an active matching booking. A valid check-in sets status to "In Use" + timestamp; a valid check-out sets status to "Available" + timestamp. Invalid attempts are rejected with an error.
US-11	As a system user, I want to log in via Google OAuth or email/password so I can access the system securely.	Any Ph.D. Student, Undergraduate Student, Lab Manager/Administrator, or Professor can authenticate via Google OAuth 2.0 or email/password. Success grants access; failure shows an error message.
US-12	As a system user, I want role-based access so I can access features according to my role.	Each authenticated user is granted access according to their assigned role (Ph.D. Student, Undergraduate Student, Lab Manager/Administrator, or Professor). Out-of-role access attempts are denied for all roles.
US-13	As a Lab Manager/Administrator, I want to manage user accounts and roles so I can control system access.	Lab Managers/Administrators can create, edit, deactivate accounts, and assign/update roles for any user. Deactivated accounts cannot log in.

6. User Requirement Specification

ID	User Requirement
URS-1	Ph.D. Students, Undergraduate Students, Lab Managers/Administrators, and Professors shall be able to view resource name, type, category, and status, and shall be able to search by keyword and filter/sort resources by category, location, availability, and status.
URS-2	Lab Managers/Administrators shall be able to add, edit, and archive resource records, including status, availability, and ownership. The system shall require valid data in required fields before saving, and shall display a validation message when data is missing or invalid.
URS-3	Ph.D. Students and Undergraduate Students shall be able to submit a booking or request specifying resource, date/time, and purpose. The system shall check for overlapping bookings and block the request if a conflict exists.
URS-4	Ph.D. Students and Undergraduate Students shall be able to view their own bookings/requests and their current status (Pending/Approved/Rejected), which shall update automatically after a Lab Manager/Administrator's decision.
URS-5	Lab Managers/Administrators shall be able to view pending requests and approve or reject each one, with status updated immediately and visible to the requester.
URS-6	Lab Managers/Administrators shall be able to log a resource issue with description, status, and date. The resource status shall automatically change to Maintenance and be blocked from new bookings.
URS-7	Lab Managers/Administrators shall be able to update maintenance records and mark issues resolved, with the resource status automatically reverting to Available.
URS-8	Lab Managers/Administrators and Professors shall be able to view the activity log (action, user, timestamp) and filter it by date, user, or action type.
URS-9	Lab Managers/Administrators and Professors shall be able to view a dashboard showing total/available resources, active bookings, and open issues, with utilization shown by category over a selectable time period.
URS-10	Ph.D. Students and Undergraduate Students shall be able to scan or enter a resource code to check in or check out a resource. The system shall verify an active matching booking; a valid check-in shall set status to "In Use" with a timestamp, a valid check-out shall set status to "Available" with a timestamp, and invalid attempts shall be rejected with an error.
URS-11	Any Ph.D. Student, Undergraduate Student, Lab Manager/Administrator, or Professor shall be able to authenticate via Google OAuth 2.0 or email/password, with success granting access and failure displaying an error message.
URS-12	Each authenticated user shall be granted access according to their assigned role, and out-of-role access attempts shall be denied for all roles.
URS-13	Lab Managers/Administrators shall be able to create, edit, and deactivate user accounts and assign/update roles for any user; deactivated accounts shall not be able to log in.

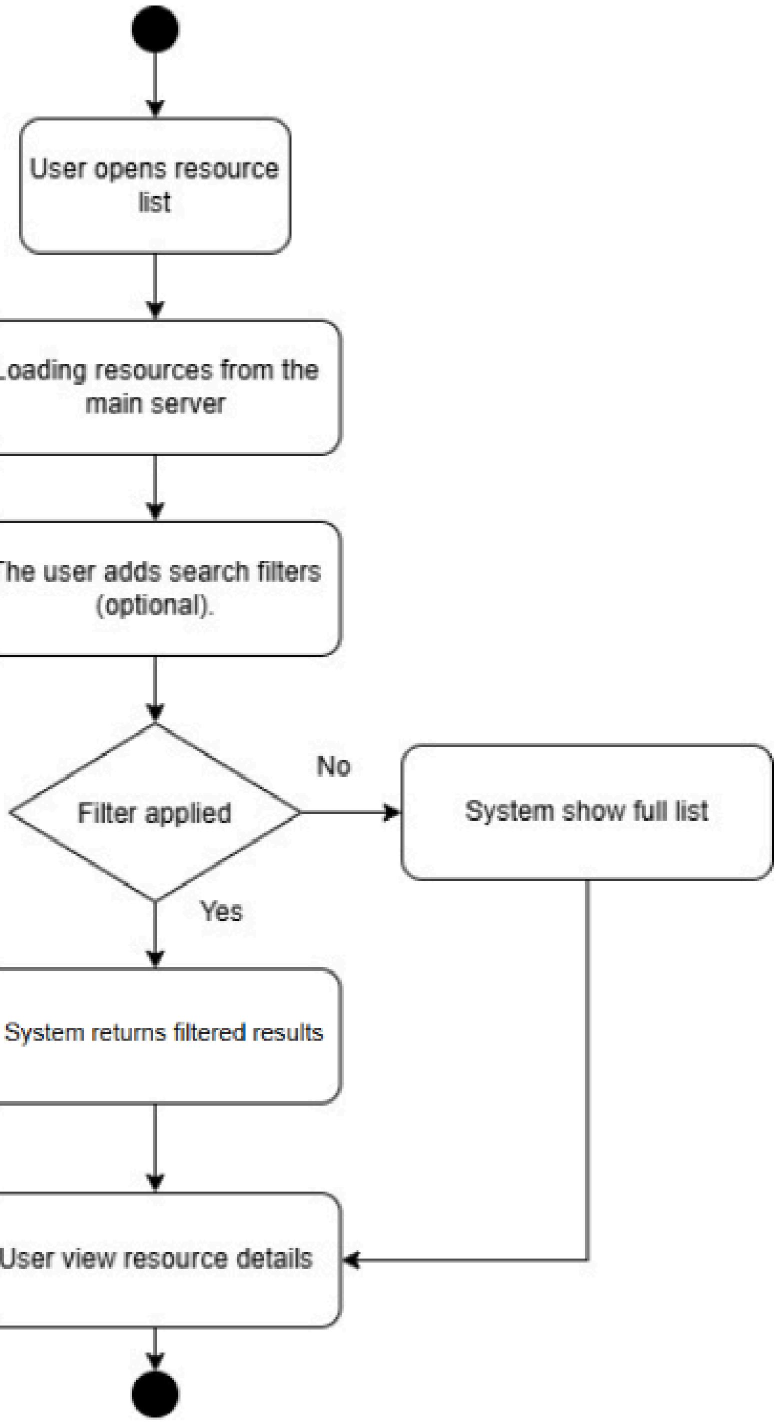
7. Activity Diagram

AD-1: Manage Resources.
This diagram covers how Lab Manager/Administrator manage resources such as adding, updating, or archiving them. We used it to decide required fields and missing-data rules, forming SRS-2.



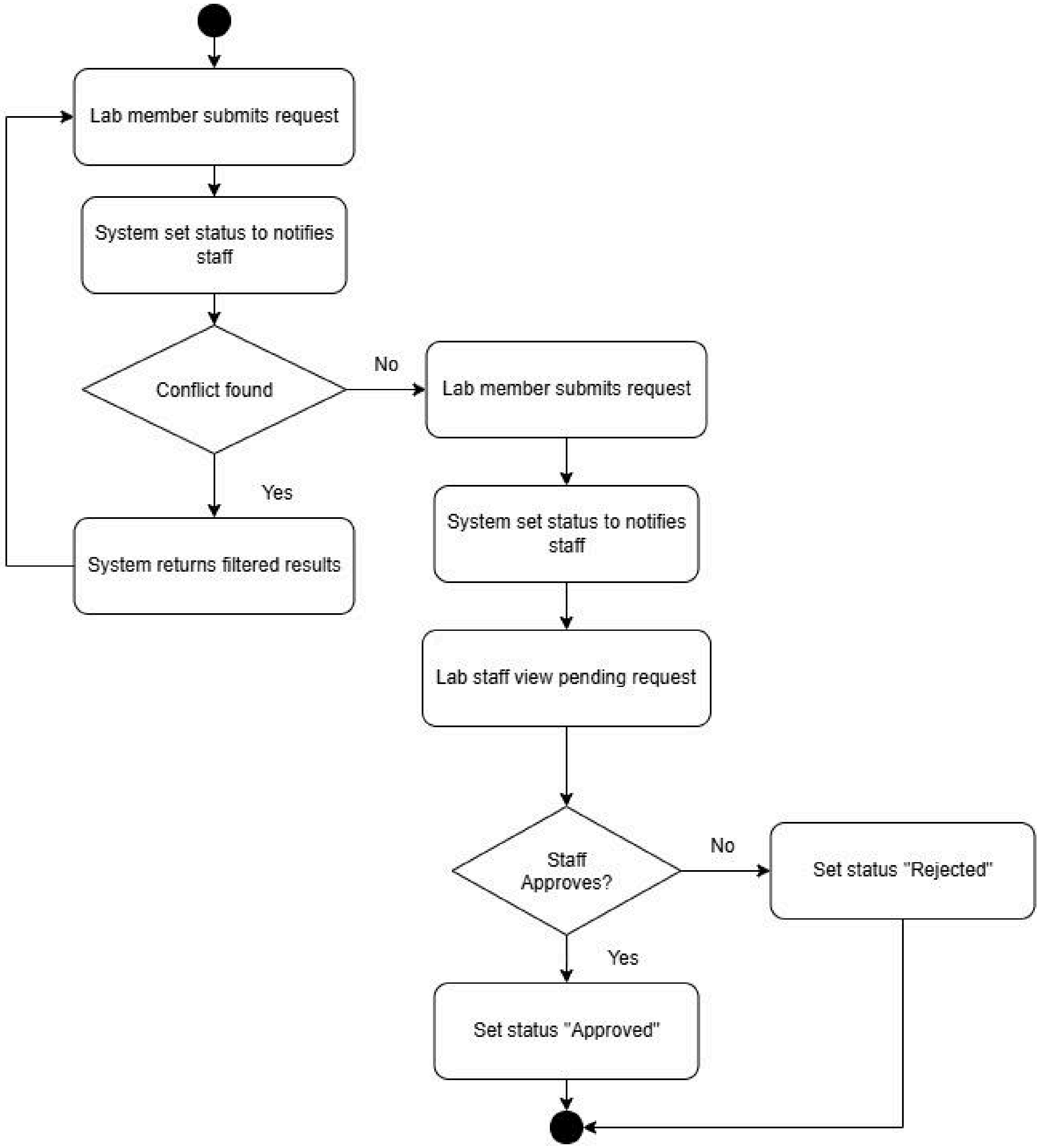
AD-2: View search resource

This diagram covers how users search and view resources, including filtering and checking details. It's mapped directly to SRS-1.



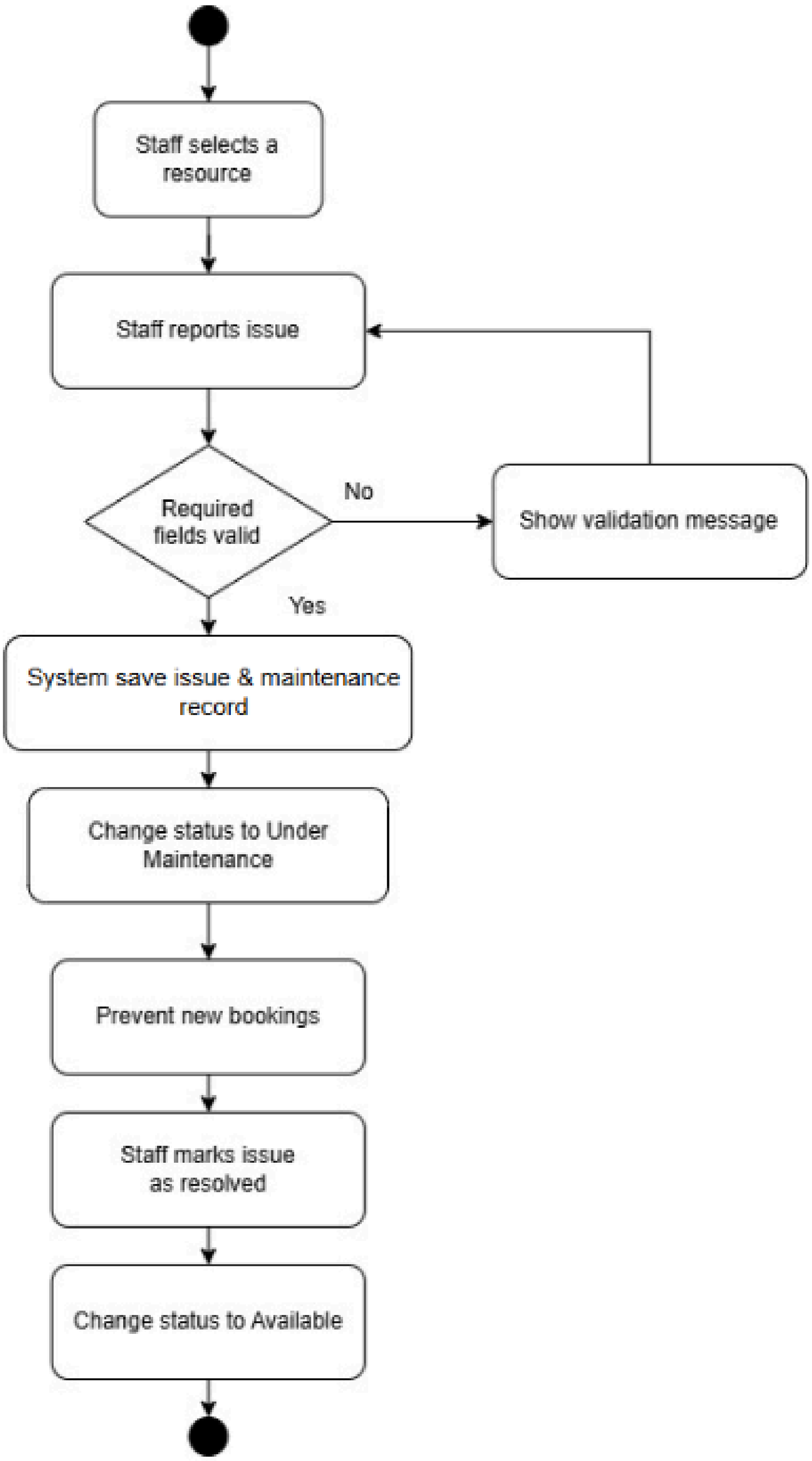
AD-3: Manage booking requests

This diagram handles the full booking process, covering both member requests and Lab Manager/Administrator approvals. It helped us shape SRS-3 especially the conflict check for double-booking.



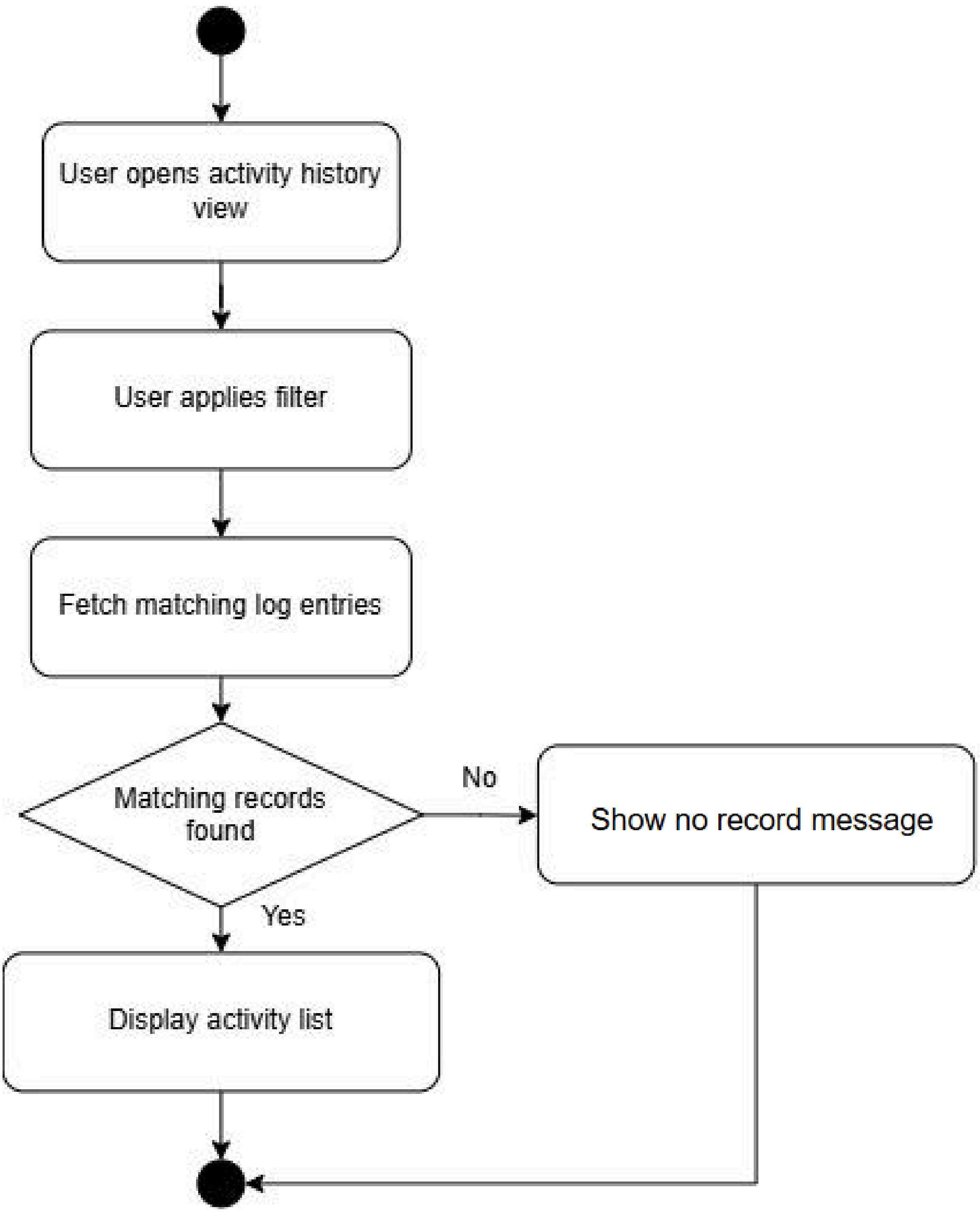
AD-4: Manage issues maintenance

This diagram covers the maintenance process for broken resources Lab Manager/Administrator reports the issue, the system blocks new bookings, and Lab Manager/Administrator resolves it when fixed. It directly supports SRS-6 and SRS-7.



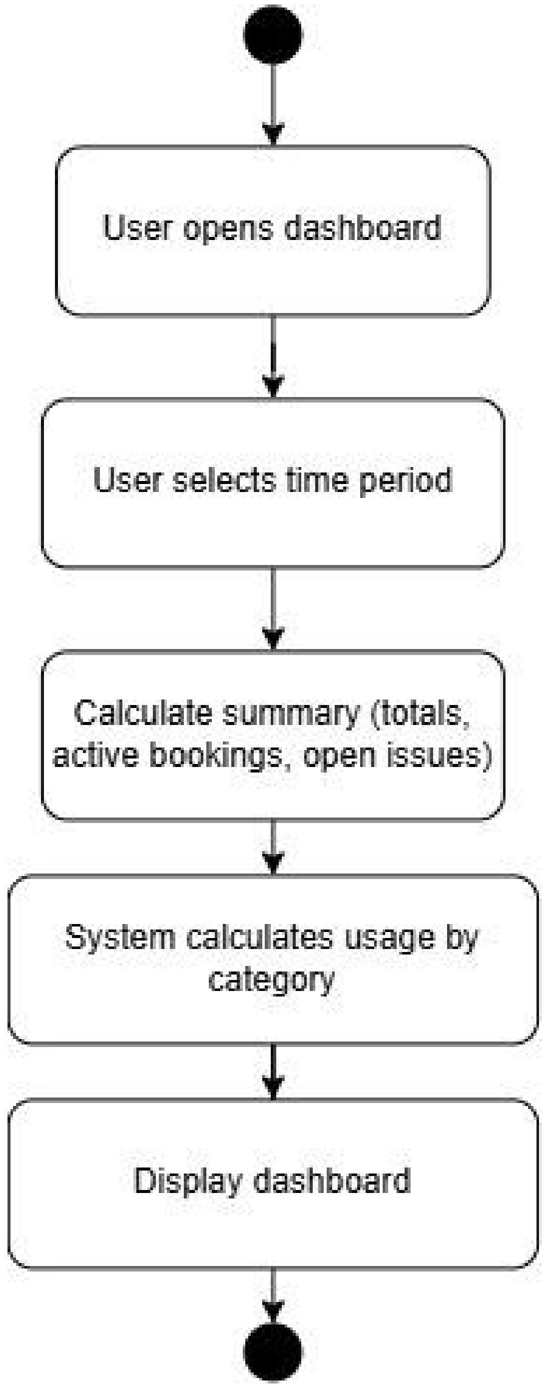
AD-5: View activity history

This diagram covers viewing and filtering the activity history log. Users can browse past events based on their search criteria, which is tied to SRS-8.



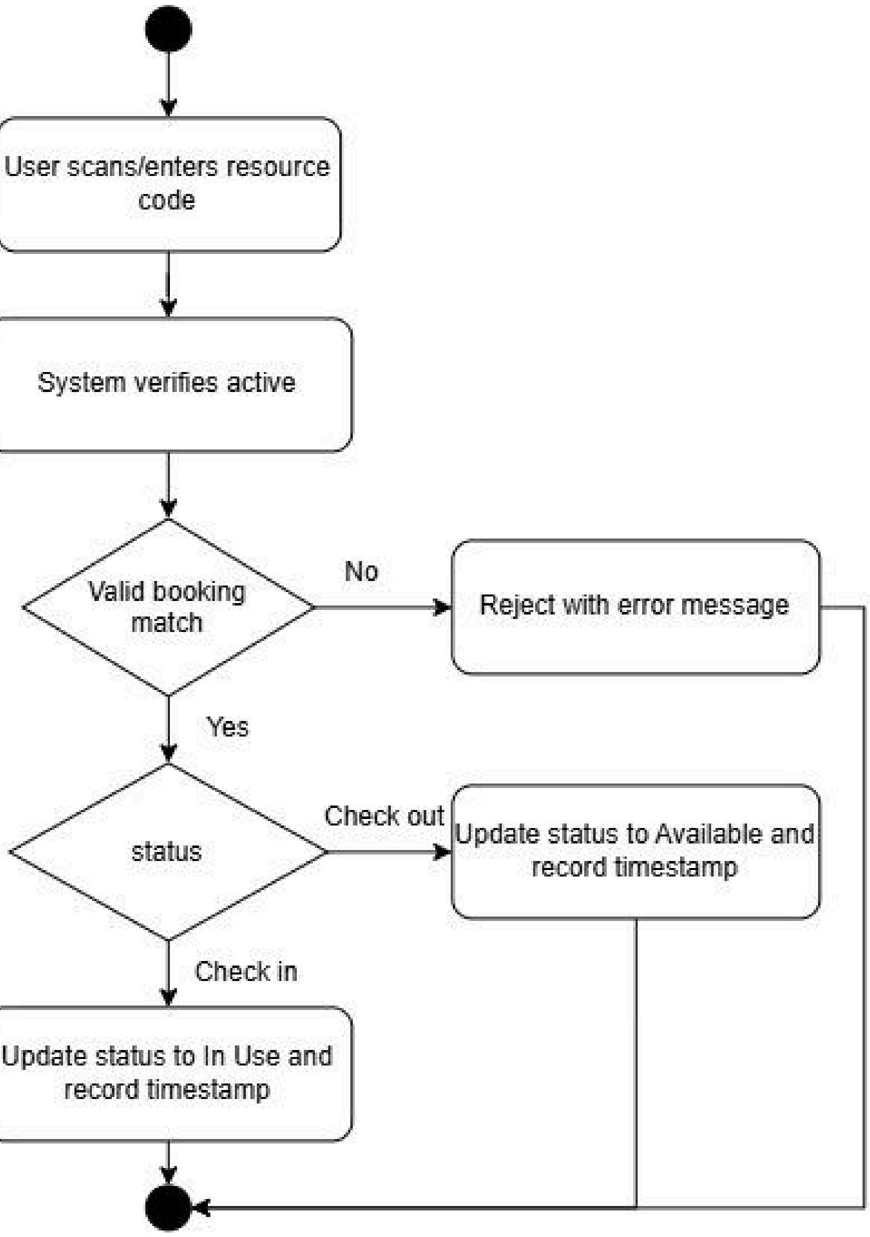
AD-6: View operations dashboard

This diagram covers how dashboard metrics are generated for a selected timeframe, including availability, active bookings, and open issues. It directly supports SRS-9.

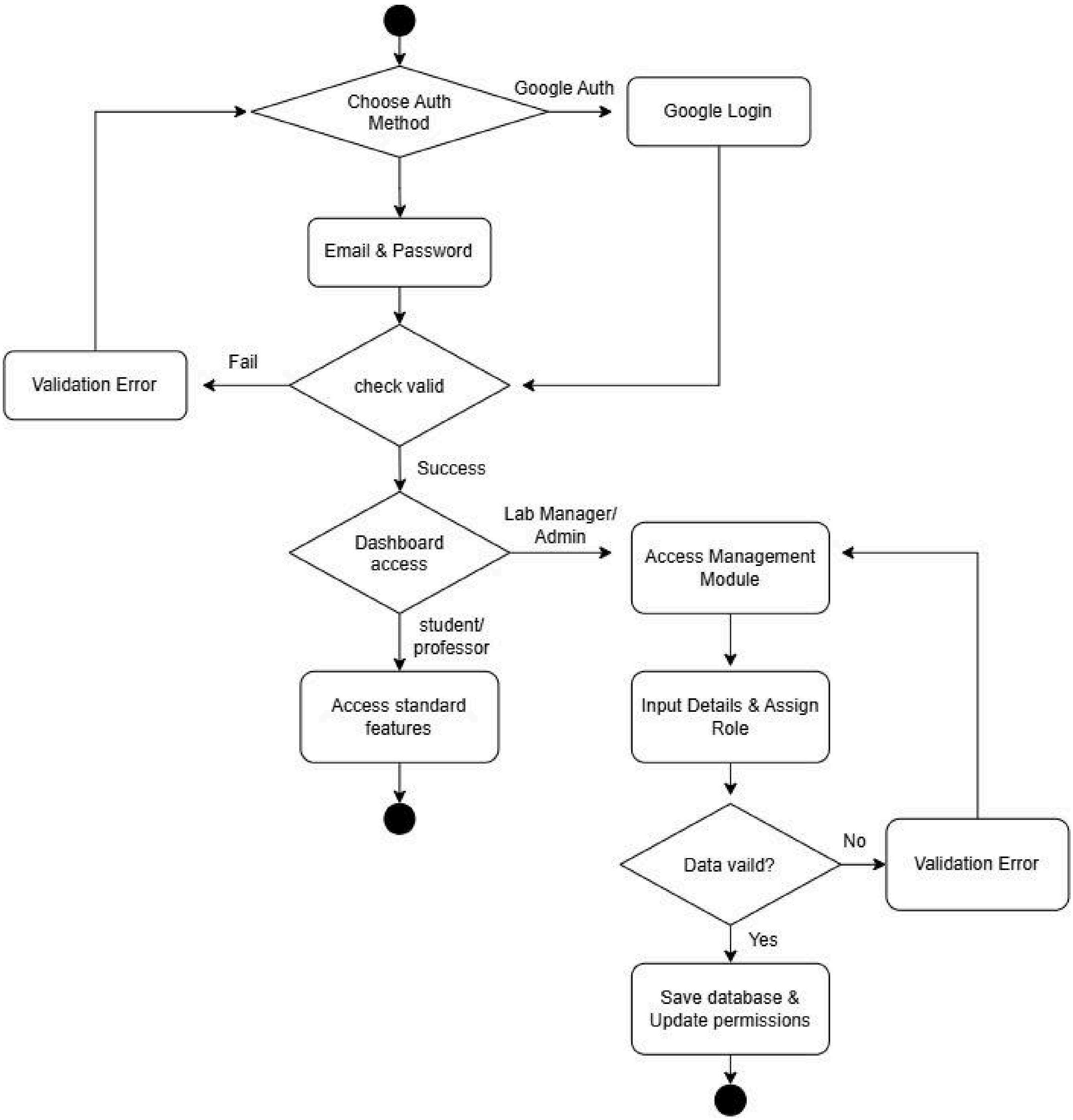


AD-7 : Check in check out

This diagram details the check-in and check-out process via resource code scanning, including booking validation and error handling for invalid attempts. It connects directly to SRS-10.



AD-8 : Authentication and User Management
This diagram covers the user authentication process via Google OAuth or email/password, as well as how Lab Managers/Administrators manage user accounts and assign roles. It directly supports SRS-11, SRS-12 and SRS-13.



8. System Requirement Specification

ID	System Requirement
SRS-1	The system shall display resource name, type, category, and status, and shall allow users to search by keyword and filter/sort by category, location, availability, and status.
SRS-2	The system shall allow only Lab Managers/Administrators to add, edit, and archive resource records, including status, availability, and ownership. The system shall require valid data in required fields before saving, and shall display a validation message and block saving when data is missing or invalid.
SRS-3	The system shall allow Ph.D. Students and Undergraduate Students to submit a booking/request specifying resource, date/time, and purpose. The system shall check for overlapping bookings and block the request if a conflict exists.
SRS-4	The system shall allow requesters to view their own bookings/requests and current status (Pending/Approved/Rejected), and shall update the status automatically after a Lab Manager/Administrator's decision.
SRS-5	The system shall allow Lab Managers/Administrators to view pending requests and approve or reject each one. The system shall update the status immediately and make it visible to the requester.
SRS-6	The system shall allow Lab Managers/Administrators to log a resource issue with description, status, and date. The system shall automatically change the resource status to "Maintenance" and block it from new bookings.
SRS-7	The system shall allow Lab Managers/Administrators to update maintenance records and mark issues resolved. The system shall automatically revert the resource status to "Available."
SRS-8	The system shall maintain an activity log (action, user, timestamp) and shall allow Lab Managers/Administrators and Professors to view and filter it by date, user, or action type.
SRS-9	The system shall display a dashboard, viewable by Lab Managers/Administrators and Professors, showing total/available resources, active bookings, and open issues, with utilization shown by category over a selectable time period.
SRS-10	The system shall provide a unique resource code for each resource. Ph.D. Students and Undergraduate Students shall scan or enter the code to check in or check out; the system shall verify an active matching booking, set status to "In Use" with a timestamp on valid check-in, set status to "Available" with a timestamp on valid check-out, and reject invalid attempts with an error.
SRS-11	The system shall authenticate users via Google OAuth 2.0 or email/password, granting access on success and displaying an error on failure.
SRS-12	The system shall grant access to features and information according to the user's assigned role, and shall deny out-of-role access attempts for all roles.

SRS-13	The system shall allow Lab Managers/Administrators to create, edit, and deactivate user accounts and assign/update roles for any user. Deactivated accounts shall not be able to log in.

9. Software Architecture

Based on the Software Requirement Specification for the VASE Laboratory Resource Management System, the recommended architecture is a **Modular Monolithic Architecture using the Model-View-Controller (MVC) pattern.** The system is implemented as a single application while separating presentation, request handling, business logic, and data management into logically independent modules.

This architecture is suitable because the system manages several related laboratory operations, including resource management, bookings and requests, maintenance, activity history, dashboard information, and check-in/check-out. Keeping these functions within one application makes the system easier to develop, test, and maintain while still keeping each function logically separated.

The architecture also supports the centralized system goal (G-0), which requires laboratory resources and operational information to be managed through one system. The main interaction between these components follows the MVC pattern:

1. The user interacts with the system through the Web Application (View).
2. Authentication & Authorization verifies the user's identity and access permissions.
3. The Controller receives the request and coordinates the appropriate system operation.
4. The Model processes the business logic and rules.
5. The Laboratory Database stores and retrieves the required data.

The Controller receives requests from the View and coordinates the appropriate Model, which applies business rules and communicates with the Laboratory Database. Authentication and Authorization controls access based on user roles and permissions. This separation supports the system’s main functions, including resource management, booking, maintenance, activity history, dashboard monitoring and check-in/check-out.

Component	Layer/Type	Description
Web Application	Presentation (View)	Provides interfaces for lab members, staff, and administrators to view resources, submit bookings, manage resources, view activity history, monitor the dashboard and perform check-in/check-out.
Resource Controller	Controller	Receives requests related to resource searching, viewing, creating, editing and updating resource information.
Booking & Check-in/Check-out Controller	Controller	Handles booking/request submission, viewing, approval or rejection, booking conflict checking, and check-in/check-out including booking validation and resource status updates ("In Use"/"Available").
Maintenance Controller	Controller	Handles resource issue reports, maintenance activities, maintenance status updates and related operations.
History & Dashboard Controller	Controller	Handles requests for viewing and filtering activity history, and retrieves/prepares operational information for the dashboard resource availability, active bookings, open issues and utilization.
Resource Model	Model	Contains business rules related to resources, including resource information, availability, status and resource validation.
Booking & Check-in/Check-out Model	Model	Contains business rules for bookings and requests booking information, status, approval decisions, overlapping booking validation plus check-in/check-out validation and the resulting resource status changes.
Maintenance Model	Model	Handles maintenance issues, maintenance records, maintenance status, and the effect of maintenance on resource availability.
History & Dashboard Model	Model	Handles activity history records (user actions, action types, timestamps) and processes data for dashboard calculations (resource availability, active bookings, open issues or utilization).
Laboratory Database	Persistence	Stores resources, bookings, requests, maintenance records, activity history and other related laboratory information.
Authentication & Authorization Module	Cross-cutting	Verifies user identity and ensures that users can access only the features and information permitted by their assigned role.

Rationale : The Modular Monolithic Architecture with the MVC pattern is selected because it is appropriate for the current scope and complexity of the VASE Laboratory Resource Management System. The system requires several related functions to work with shared data and business rules, making a single application more practical than a distributed architecture such as Microservices. This approach reduces the complexity of communication and deployment while keeping the system manageable. It also provides sufficient flexibility to extend the system with additional modules in the future without introducing unnecessary architectural complexity.

10. Data Storage Technology

The recommended data storage technology is MySQL, a Relational Database Management System (RDBMS), used as the centralized data store for the laboratory resource management system.

Component	Data Storage Technology	Description
Resource Data	MySQL	Stores resource information such as resource code, name, category, location, availability, status and responsible person.
Booking and Request Data	MySQL	Stores booking and request information, including the requester, resource, date/time, purpose and current status.
Maintenance Data	MySQL	Stores reported issues, maintenance activities, descriptions, dates and maintenance status.
Activity History	MySQL	Stores system actions, the users who performed them and timestamps.
User and Access Data	MySQL	Stores user accounts and role information for authentication and access control.
Dashboard Data	MySQL Queries	Provides data for calculating resource availability, active bookings, open maintenance issues and resource utilization.

Rationale : A relational database is appropriate because the system contains structured and related data, including resources, bookings, requests, maintenance activities, users and activity history. It also supports data integrity and consistency, such as preventing overlapping confirmed bookings and maintaining accurate resource availability.

Rationale : A single centralized MySQL database is suitable because the system aims to manage laboratory resources and operational information in one centralized system.

11. Development Environment

The recommended development and deployment environment uses **Docker** to provide a consistent environment for the application and database. The system uses **HTML, JavaScript and CSS** to develop the user interface, while the **MVC application** handles system operations and communicates with the **MySQL** database for data storage.

Component	**Development Environment**	**Description**
User Interface	HTML & CSS & JavaScript	Provides the web interface for lab members, staff, and administrators to access system functions.
Application Server	Docker Container (Python)	Runs the MVC application and handles requests from the user interface.
Database Server	MySQL in Docker Container	Stores centralized laboratory data, including resources, bookings, requests, maintenance records, users, and activity history.
Docker Compose	Container Orchestration	Starts and manages the application and MySQL database containers together.
Development Machine	Local Computer & Docker Desktop	Used by developers to implement, test and debug the system.
Web Browser	Client Environment	Used by lab members, staff, and administrators to access the web interface.
Backend Framework & Libraries	Python	Handles application logic, routing, and controller operations for the MVC application.

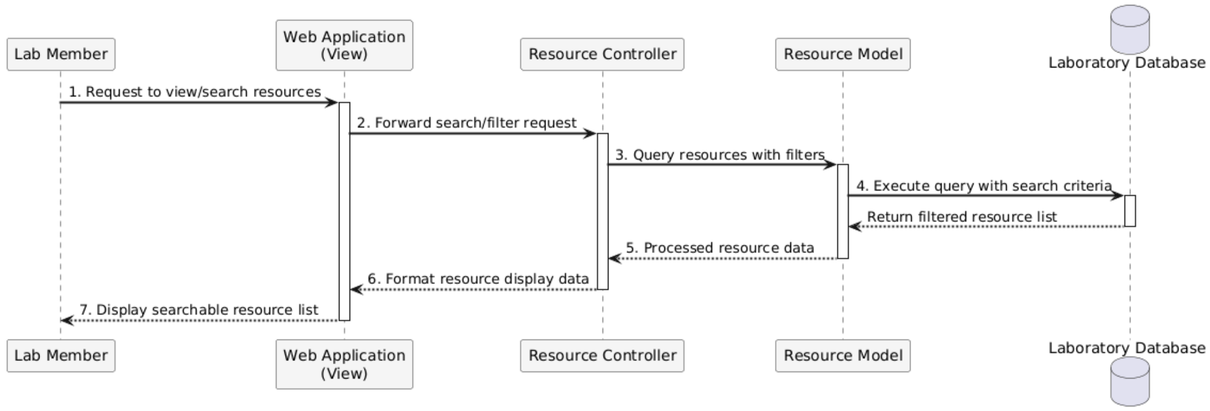
Rationale: The development environment separates the application and database into different containers while allowing them to work together as one system. This provides a consistent environment for development and deployment while keeping the application and database components logically separated.

12. Sequence Diagrams for All SRS

** Lab members include Ph.D. Students and Undergraduate Students**

SQD-1 : User Views/Searches Resources

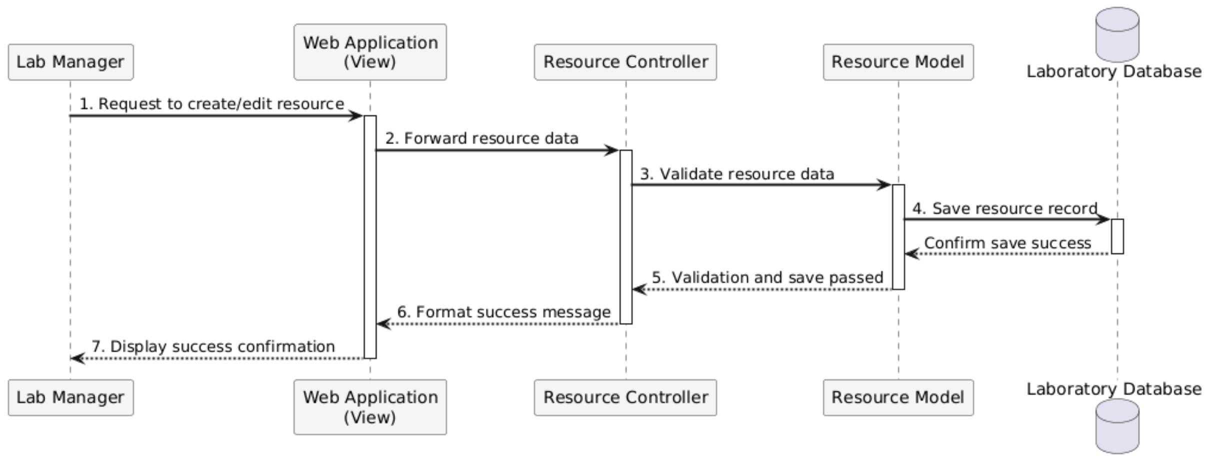
Traced requirements : SRS-1



SRS-1

SQD-2 : Administrator Creates/Edits Resource (Success Path)

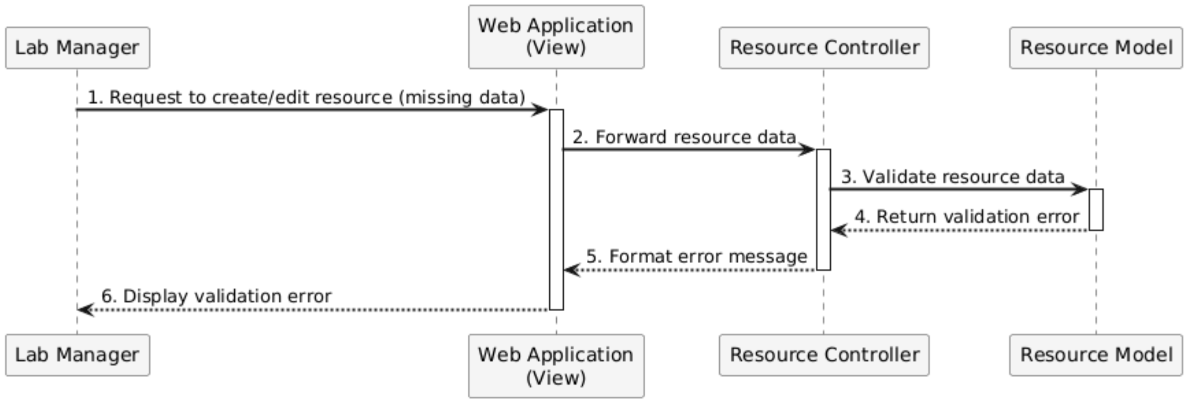
Traced requirements : SRS-2



SRS-2

SQD-3 : Administrator Creates/Edits Resource (Error Path)

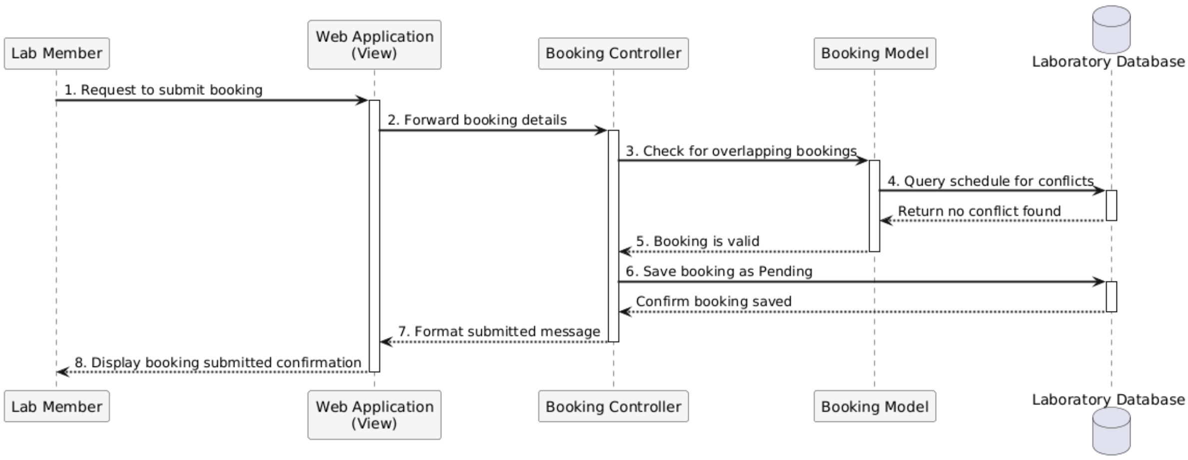
Traced requirements : SRS-2



SRS-2 (Error Path)

SQD-4 : Member Submits Booking (Success Path)

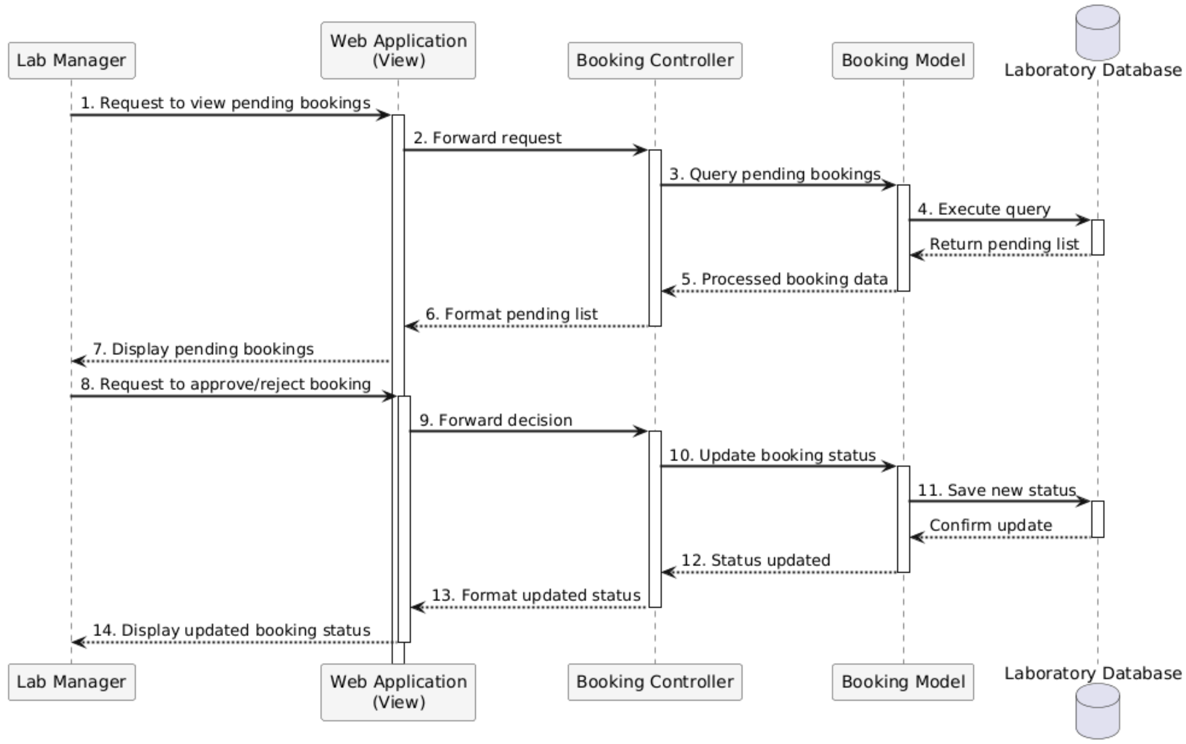
Traced requirements : SRS-3 and SRS-4



SRS-3, SRS-4

SQD-5 : Staff Reviews and Approves Booking

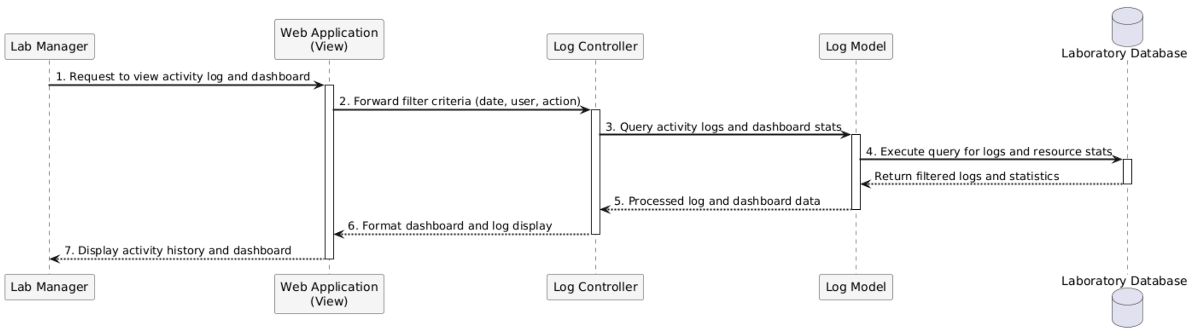
Traced requirements : SRS-5



SRS-5

SQD-6 : Staff Views Activity Log and Dashboard

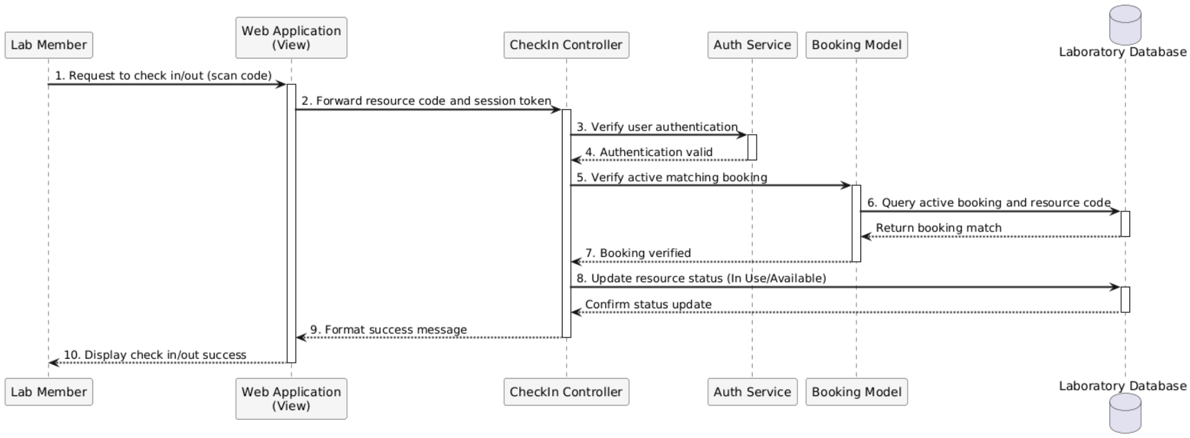
Traced requirements : SRS-8, SRS-9



SRS-8, SRS-9

SQD-7 : Member Checks In Resource

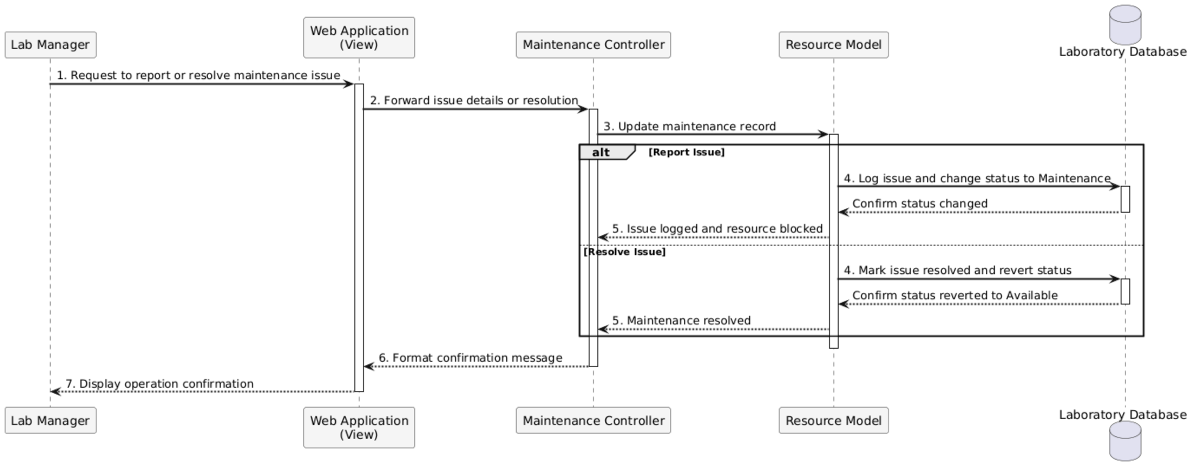
Traced requirements : SRS-10, SRS-11



SRS-10, SRS-11

SQD-8 : Admin Resolves Maintenance Issue

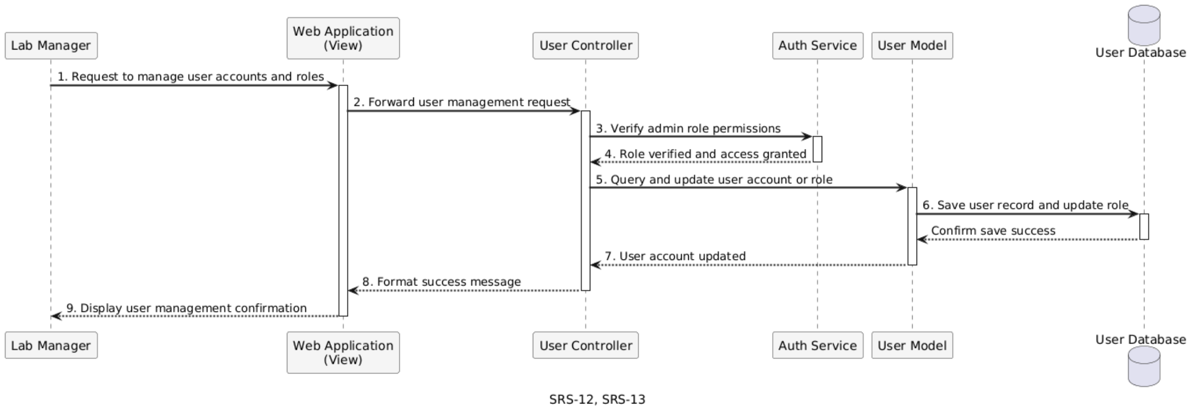
Traced requirements: SRS-6, SRS-7



SRS-6, SRS-7

SQD-9: Admin Manages Users and Roles

Traced requirements : SRS-12, SRS-13



13. Taceability Matrix

Sub-Goal	Use Case	User Story	URS	Activity Diagram	SRS
SG-1	UC-1 View /Search Resources	US-1	URS-1	AD-2	SRS-1
SG-1	UC-2 Manage Resources	US-2	URS-2	AD-1	SRS-2
SG-2 , SG-5	UC-3 Manage Bookings & Requests	US-3, US-4, US-5	URS-3, URS-4, URS-5	AD-3	SRS-3, SRS-4, SRS-5
SG-3	UC-4 Manage Issues & Maintenance	US-6, US-7	URS-6, URS-7	AD-4	SRS-6, SRS-7
SG-4	UC-5 View Activity History	US-8	URS-8	AD-5	SRS-8
SG-4	UC-6 View Operations Dashboard	US-9	URS-9	AD-6	SRS-9
SG-5	UC-7 Check In/Check Out Resource	US-10	URS-10	AD-7	SRS-10
SG-6	UC-8 Maintain System & Access	US-11, US-12, US-13	URS-11, URS-12, URS-13	AD-8	SRS-11 ,SRS-12, SRS-13

From SRS to Sequence Diagram

SRS	SQD
SRS-1	SQD-1
SRS-2	SQD-2
SRS-3	SQD-3
SRS-4	SQD-4
SRS-5	SQD-5
SRS-6	SQD-4
SRS-7	SQD-8
SRS-8	SQD-6
SRS-9	SQD-6
SRS-10	SQD-7
SRS-11	SQD-7
SRS-12	SQD-9
SRS-13	SQD-9